

VLA drone lab workbook

Mac and workstation edition | 26 September 2026

How to use this workbook

This is the current practical route through all 12 labs. Start with the local exercises on your Mac, learn to interpret each result, then move to the simulator and model integration environments when they are available. The original 84-page thesis guide remains the theory and literature reference. This workbook supersedes its lab commands and completion checklists.

The aim is to explain and build the observation-to-action loop, then extend it to a complete survey mission. Running a command is only part of completing a lab. Before each experiment, write a prediction; afterwards, explain one observation and one limitation using saved evidence.

Choose the environment

Environment	Work	Current evidence
Mac CPU	Labs 1–5, offline preparation for 6–10, Lab 11 logic, Lab 12 data tests	Laptop workflow executed on M4; research logic still to implement
Ubuntu 22.04 simulator host	Labs 6–8 and mission execution	PX4 v1.16.0 and Gazebo Harmonic are candidate pins; integration smoke tests pending
CUDA workstation	Labs 9–10 and larger Lab 12 training	Supplied scripts require CUDA; model runtime unverified
Apple GPU experiment	Optional future adaptation of 9–10	MPS support must be implemented and measured; current scripts do not support it
Orin and aircraft	Deployment and real transfer after Lab 12	Require separate equipment and experiments

Your MacBook Air M4 with 24 GB unified memory is sufficient for the CPU route. The measured 33 GiB of free storage on 26 September 2026 is a snapshot; check again before simulator installations or model downloads. No VM, model download or hardware connection is needed for Labs 1–5.

Use an Ubuntu lab workstation through remote desktop for the simulator. Run Gazebo, PX4, camera recording and MAVSDK on that same Ubuntu host. Opening a terminal on your Mac does not move those processes onto the remote host. A VM or native macOS port is an optional compatibility project with separate evidence requirements.

Learning order and prerequisites

Keep the original lab numbers so references in the thesis guide remain useful. The recommended order is **1–5, 11A, 6–8, 11B, 9–10, 12**. Lab 11A means the offline state-machine milestone; Lab 11B means integrating it with the validated simulator bridge.

Lab	Subject	Needs	Evidence to save
1	First episode	Mac setup	Three trajectories and environment record
2	Coordinates and command anchoring	Lab 1	Worked command and geometry checks
3	Demonstrations and information boundaries	Lab 2	Validated dataset and one annotated sample
4	Train the tiny policy	Lab 3	Checkpoint, loss history and selected epoch
5	Evaluate and explain failures	Lab 4	Paired validation results and failure replay
6	Default PX4 and Gazebo	Labs 1–5; Ubuntu	Restartable simulator and version record

Lab	Subject	Needs	Evidence to save
7	Camera and field geometry	Lab 6	Images, timestamp checks and axis calibration
8	Flight commands and bridge	Labs 6–7	Square telemetry and separate bridge fault evidence
9	SmoIVLA interface	Labs 3–5; CUDA for runtime	Tensor, gradient and reload checks
10	SmoIVLA development	Lab 9	Checkpoint, validation behaviour and timing
11	Detour and resumption	Lab 5 for A; Lab 8 bridge for B	Event replay, recovery and coverage tests
12	Unified mission model	Labs 10 and 11B	Grouped missions and complete-system comparisons

Start the Mac environment

Open a terminal in VLA_Drone_Starter_Kit. Use your IDE's "Open in Integrated Terminal", or type `cd` and drag that folder into Terminal. This handles spaces in the folder name. Confirm that `ls` shows `litterlab`, `integration`, `tests` and `requirements.txt`.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m integration.lab_tools doctor
python -m unittest discover -s tests -v
```

Expect 22 passing tests in this revision. Our CPU verification used Python 3.14.6, NumPy 2.5.3 and Pillow 12.3.0. To reproduce those package versions with that Python version, use `requirements-cpu-tested.txt`. The broader `requirements.txt` is suitable for the supported CPU exercise environments; matching test results is the check, not merely completing installation.

Each new terminal needs `source .venv/bin/activate`. In VS Code, select `.venv/bin/python` as the interpreter. Keep the CUDA and simulator environments separate; this CPU environment does not install their dependencies.

Evidence and run folders

All commands below run from the kit root unless explicitly stated. Use a fresh output folder per run. If one already exists, choose a new name and use it consistently in dependent commands. Existing run folders are deliberately protected from accidental overwrites.

```
mkdir -p notes
cp labs/WORKSHEET.md notes/lab01.md
python -m integration.lab_tools doctor > notes/environment.json
python -m pip freeze > notes/cpu-packages.txt
```

Copy the worksheet for each lab. Mark work as **not started**, **in progress**, **passed with evidence**, or **blocked by environment**. A local preparation exercise for a workstation lab does not earn its runtime pass.

Corrections to the original guide

- The current suite has 22 tests; the original guide reports its historical ten-test run.
- Tiny-policy evaluation defaults to validation scenes. Use `--split val` for development and explicit `--split test` to reproduce the original 30/30 and 24/30 results.
- The dataset validator checks the cache against images and records. The trainer performs that check before training.
- Swapped and empty task interventions require a learned model; trials record the actual input instruction and retain the original requested goal for scoring.
- Labs 11 and 12 specify software to build. Their proposed filenames are not installed commands.
- Passing CPU tests does not validate PX4, Gazebo, SmoIVLA, Orin or real flight.

Lab 1 Run and explain your first episode

Where: Mac CPU. **Before starting:** complete the workbook environment setup. **Theory:** original guide pages 5–12 and 22. **Status:** runnable teaching exercise.

Goal

Recognise the loop: acquire an image, choose a horizontal movement, turn it into an absolute position target, simulate motion, then acquire another image. Identify which files contain observations, commands and results.

Predict before running

Blue represents the can, red the bottle and yellow the paper. With the field held fixed, should a different requested category change the image at the start, the first movement, both, or neither? Write your prediction before comparing the three runs.

Run the experiment

```
python -m unittest discover -s tests -v
python -m litterlab demo --seed 7 --target can --out runs/lab01-can
python -m litterlab demo --seed 7 --target bottle --out runs/lab01-bottle
python -m litterlab demo --seed 7 --target paper --out runs/lab01-paper
python -m integration.lab_tools episode runs/lab01-can
```

On macOS, open the replay and trajectory:

```
open runs/lab01-can/replay.gif
open runs/lab01-can/trajectory.png
```

If your viewer shows only one GIF frame, use a browser or an animation-capable viewer. The per-step PNGs are also in frames/.

Inspect the evidence

1. Read `result.json`. The can run should succeed in nine steps and 4.5 simulated seconds with the default dynamics.
2. Compare `frames/0000.png` across all three runs. The initial field and pose use the same seed; the requested task changes the control decision.
3. Compare the first `action_delta_ne_m` in each `steps.jsonl`. Describe the movement with north, south, east and west.
4. Open `map_local.json`. Its coordinates are local NED metres. This file is not a GPS map or GeoJSON.
5. Notice that the replay includes an added crosshair, while saved model-input frames do not. Presentation overlays should not leak into model inputs.

Troubleshooting

- No module named `litterlab`: confirm the terminal is in the kit root and the interpreter is the project `.venv`.
- `FileExistsError`: choose a fresh output folder rather than deleting a result you still need.
- A different can result: retain the command and environment record; inspect whether the seed, model or source changed.

Completion check

Save the three trajectories, the test log and your environment record. Explain in five sentences why the initial scene stays fixed while the trajectories differ. You pass when you can locate an observation, action, accepted target and outcome in the files.

Optional extension

Repeat only the can run with seed 8 in a new folder. Explain why this changes the scene as well as the resulting path. Do not describe either run as a real-drone experiment.

Lab 2 Trace coordinates and command anchoring

Where: Mac CPU. **Before starting:** Lab 1 can episode. **Theory:** guide pages 9–10, 19–20 and 23. **Status:** runnable teaching exercise.

Goal

Distinguish pixels, relative displacement, absolute position targets and measured position. Explain why one prediction must be anchored once to the pose associated with its observation.

Predict before running

For the ideal 96-pixel camera, the centre is (48, 48), focal length is 48 pixels and height is 5 m. A target at (60, 36) lies north and east. Compute its ground offset and then scale the vector to a maximum length of 0.5 m. Does bounding each component separately give the same vector-length limit?

Run the checks

```
python -m integration.lab_tools episode runs/lab01-can
python - <<'PY'
from litterlab.core import pixel_to_offset, bounded
offset = pixel_to_offset(60, 36)
print('Ground offset NE in metres:', offset)
print('Bounded command NE in metres:', bounded(offset))
PY
```

The worked offset is [1.25, 1.25] m. Its bounded command is approximately [0.3536, 0.3536] m. Both components can be below 0.5 while their combined vector exceeds 0.5, which is why the norm matters.

Inspect one actual command

1. Take the first capture position and action from the Lab 1 can run. Add north to north and east to east by hand.
2. Compare with `accepted_position_ne_m`. The first seed-7 target is approximately [-0.1204, -0.0103] m.
3. Compare that target with the next row's measured position. The simulated vehicle needs time to approach a target; an accepted command is not proof that it has been reached.
4. Read `observe`, `classical`, `bounded` and apply in `litterlab/core.py`. Then read `rollout` in `litterlab/__main__.py`.
5. Explain why the logged down coordinate is -5 for a vehicle 5 m above the local origin.

Change one condition

In a scratch Python session, call `bounded([3, 4])`, then `bounded([float('nan'), 0])`. The first should have norm 0.5; the second should raise an error. Explain why silently passing NaN into a controller would make subsequent computations unreliable.

The geometry assumes a level nadir camera with north at image top. Write down which assumptions break when yaw, roll, pitch, terrain slope or camera mounting changes. Do not apply this projection unchanged to Gazebo imagery.

Completion check

Save your hand calculation, helper output and axis sketch. Explain the capture anchor, vector bound, boundary clipping and measured response. Describe the error caused by adding the same displacement again on every command resend.

Troubleshooting

If the helper reports that anchor plus action differs from the accepted target, inspect whether the simulator clipped the target at the field boundary. That can be valid behaviour. A sign mismatch at an interior point requires checking axis order and pixel direction.

Lab 3 Collect and audit demonstrations

Where: Mac CPU. **Before starting:** Lab 2. **Theory:** guide pages 24–25 and 43–48. **Status:** runnable teaching exercise.

Goal

Trace one supervised learning example from its image to its task, state and action label. Separate deployable inputs from expert-only information and keep related samples in the same split.

Predict before running

The expert can access exact object positions. The learner receives image-derived features, a requested category and velocity. What would happen to measured performance if the learner were given the target's true coordinates? Why could random frame splitting make a model look better than it is?

Collect and validate

```
python -m litterlab collect --out runs/lab03-data
python -m integration.data --data runs/lab03-data
python -m integration.lab_tools episode runs/lab03-data/train_0000
```

Expected collection: 150 training, 30 validation and 30 test episodes, zero expert failures and 1,569 samples. Validation reports 1,144 training, 210 validation and 215 test samples. Counts refer to the supplied default generator.

Follow one record

1. Open `train_0000/images/0000.png` and the first row of `steps.jsonl`. Locate the requested category visually and predict the action signs.
2. Identify the seven state values: north velocity, east velocity, height, roll, pitch, sine yaw and cosine yaw. Distinguish this from the 14-element tiny-policy feature vector.
3. Verify that timestamps start at zero and advance by 0.5 s. Each action belongs to the observation recorded on that row.
4. Read `manifest.json`. The seed ranges start at 0, 100000 and 200000 for train, validation and test. Keep entire episodes together.
5. Open `evaluator_only.json` to understand its contents, then inspect the loader in `integration/data.py`: those coordinates must not become policy inputs.

Deliberately break a copy

Copy `runs/lab03-data` to a fresh scratch directory, change the first action in one training `steps.jsonl` to `[5.0, 0.0]`, and run the validator on the copy. It should reject the bound violation. Restore the scratch copy, change the action slightly within bounds without rebuilding its cache, and confirm the validator detects the disagreement with `samples.npz`. Keep the original dataset intact.

Completion check

Save the validator output and an annotated record showing the image, task, state and label. Explain why `samples.npz` is a derived cache, why evaluator truth is separate, and why a split seed is only sufficient for this controlled generator. Real missions also need layout, collection-session and physical-item grouping.

Troubleshooting and extension

A missing or stale cache requires regenerating a complete teaching dataset in a fresh directory. Do not dismiss the error by removing checks. For a quick second dataset, use `--train 3 --val 3 --test 3`; label it a small workflow test and report its actual counts.

Lab 4 Train and inspect the tiny learner

Where: Mac CPU. **Before starting:** validated Lab 3 dataset. **Theory:** guide pages 25–26. **Status:** runnable teaching exercise.

Goal

Understand how supervised action labels train a small model and how validation selects a checkpoint. Separate action agreement from successful behaviour.

Predict before running

What should happen to training loss as the optimiser updates the model? Can lower action error guarantee that the vehicle reaches the right object? Write one reason a learned policy might enter states missing from expert demonstrations.

Train and inspect

```
python -m litterlab train --data runs/lab03-data --out runs/lab04-tiny
python -m integration.lab_tools training runs/lab04-tiny
```

With defaults, training uses 200 epochs, batches of up to 128 samples, learning rate 0.002 and seed 0. The default dataset yields 1,800 optimiser updates. The original best validation normalised action MSE is approximately 0.0223172; exact equality across different environments is not a universal requirement.

Read the model and its evidence

1. Read `TinyPolicy` in `litterlab/learning.py`: 14 inputs, two hidden layers of 64 units and two action outputs.
2. Count the 14 inputs: nine colour-centroid/presence features, three category indicators and two velocity values.
3. Read `config.json` and `history.json`. Identify the first, last and best validation epochs. The last epoch need not be the saved checkpoint.
4. Explain why labels are divided by 0.5 for training and predictions multiplied by 0.5 for execution. MSE in these normalised units is not an error in metres.
5. Explain what `policy.npz` stores. It is a NumPy checkpoint; it contains no pretrained vision or language backbone.

A controlled comparison

Train a second run with `--epochs 20` and a fresh output directory. Hold the dataset and seed fixed. Compare the two loss histories using the helper. Write a prediction about their closed-loop performance before Lab 5 tests it. Do not select the better model from test scenes.

For plotting practice, plot `train_mse_normalised` and `val_mse_normalised` against epoch using a tool you already know. Keep the raw JSON next to the plot and label both axes and the chosen checkpoint.

Completion check

Save the checkpoint, configuration, history and selected epoch. Explain episode, sample, batch, epoch, optimiser step and evaluation trial using this run's numbers. State which data changes the weights and which data selects the checkpoint.

Troubleshooting

The trainer validates data before creating its output directory. Resolve cache or record inconsistencies first. For unexpected loss, check the dataset path, seed, action scale and package versions. More epochs will not repair an incorrect interface.

Lab 5 Evaluate behaviour and investigate failures

Where: Mac CPU. **Before starting:** Lab 4 checkpoint. **Theory:** guide pages 27–28 and 55. **Status:** runnable teaching exercise.

Goal

Compare policies on matched starts, keep validation separate from final testing, and explain a failure from a replay. Interpret language interventions within this restricted task.

Predict before running

Will the learned policy outperform the colour-based controller in a world where colour identifies the target exactly? If you give the model a different requested category but keep the original scoring goal, what should happen to success?

Run validation comparisons

```
python -m litterlab evaluate --split val --out runs/lab05-classical
python -m litterlab evaluate --split val \
  --model runs/lab04-tiny/policy.npz --out runs/lab05-learned
python -m litterlab evaluate --split val --language swapped \
  --model runs/lab04-tiny/policy.npz --out runs/lab05-swapped
python -m litterlab evaluate --split val --language empty \
  --model runs/lab04-tiny/policy.npz --out runs/lab05-empty
python -m integration.lab_tools compare \
  runs/lab05-classical runs/lab05-learned \
  runs/lab05-swapped runs/lab05-empty
```

The helper rejects mismatched ordered seeds or requested targets. It lists failed trials for replay. Validation results need not equal the guide's historical test results.

Diagnose rather than guess

Choose a failed validation trial from the helper output. Substitute its integer seed and category in this command; SEED and CATEGORY are placeholders:

```
python -m litterlab demo --model runs/lab04-tiny/policy.npz \
  --seed SEED --target CATEGORY --out runs/lab05-failure
```

This demo replays the normal-instruction policy. It does not reproduce a swapped/empty trial. Use a failure from the normal learned run for this exercise. If that run has no failures, inspect its worst final error and explain why it still passed.

Compare the image, proposed action, accepted target and trajectory. Classify the observed cause: wrong direction, oscillation, final error, excessive speed or timeout. Success requires being within 0.20 m and below 0.18 m/s for two successive decision ticks, within 60 steps.

Change dynamics on validation scenes

Repeat the normal learned evaluation with `--response 2` in a fresh folder. This changes the simulator's velocity-response time constant. It does not inject camera or inference latency. Keep seeds fixed and explain any changed failures.

Final testing after the protocol is frozen

Repeat the four evaluation commands with `--split test` and fresh folders. The supplied seed-0 reference checkpoint yields 30/30 classical, 24/30 learned, 0/30 swapped and 0/30 empty on the original 30 test scenes. These are reference results, not expected validation scores or general language-understanding evidence.

Once test failures inform a model change, that test set is no longer untouched. Develop against validation and reserve a new test set for later research claims.

Completion check

Save paired summaries, trial records and one normal-policy failure analysis. Explain why changed-task failure can be consistent with following the changed task. Report the sample size and training-seed count. A single 30-scene run does not measure training variability.

Lab 6 Establish a repeatable flight simulator

Where: Ubuntu 22.04 lab host. **Before starting:** Labs 1–5 and access to the host. **Theory:** guide pages 29 and 34. **Status:** supplied integration procedure; runtime validation pending.

Goal

Run the real autopilot against a simulated vehicle and document an environment that can be restarted. Identify what PX4 controls, what Gazebo simulates and what the companion program requests.

Mac preparation

Draw three boxes for PX4, Gazebo and the companion Python process. Add arrows for simulated sensors, actuator commands, telemetry and offboard setpoints. Read `integration/prepare_gazebo.py` and `docs/PX4.md`. This preparation is useful locally; completing it does not establish a working simulator.

The workbook route runs the simulator stack on one Ubuntu machine. Use your Mac for editing and remote desktop. Native macOS Gazebo Classic instructions do not establish compatibility with this Harmonic exercise. Keep an experimental VM or native port in a separate environment record.

Predict before running

The CPU teaching simulator uses a kinematic point vehicle. Which new failure modes might appear with a real estimator and autopilot? Explain why passing geometry tests cannot validate sensor startup or vehicle stability.

Set up on the Ubuntu host

Have the lab engineer review the dependency setup on the designated machine. The setup script changes system packages. In a new checkout directory:

```
git clone --recursive --branch v1.16.0 \
  https://github.com/PX4/PX4-Autopilot.git
cd PX4-Autopilot
bash Tools/setup/ubuntu.sh
```

Follow any reboot instructions. Then, from that checkout:

```
git rev-parse HEAD
git submodule status
cat /etc/os-release
gz sim --versions
make px4_sitl gz_x500
```

PX4 v1.16.0 and Gazebo Harmonic are candidate pins for this package. Record what actually installed; retain build errors rather than silently switching versions. Use the matching official Ubuntu setup documentation linked in the workbook index.

Inspect and restart

1. Confirm the default world and x500 load. Save the simulator view and PX4 console log.
2. Check that simulated sensor messages and position readiness appear. Record any missing sensor or estimator warning.
3. Close the simulator cleanly. Start it again from a fresh terminal with the same command.
4. Record the OS, PX4 commit, submodules, Gazebo version and launch command in `notes/lab06.md` on the host. Retain a copy alongside the Mac notes.

Troubleshooting

Build failure: inspect the first compiler or dependency error. Missing models: verify submodules and resource paths. Rendering failure: inspect the host's graphics setup and remote desktop support. Fix the default world before generating the custom one.

Completion check

Pass only when the default simulator starts twice from recorded instructions and provides usable sensor/vehicle state. Save logs from both runs. If no Ubuntu host is available, record the lab as blocked by environment and continue Lab 11A locally.

Lab 7 Verify camera images and ground geometry

Where: Ubuntu simulator host; offline XML preparation on Mac. **Before starting:** Lab 6 default simulator. **Theory:** guide pages 30–31 and 40–42. **Status:** generated XML tested structurally; rendering and calibration pending.

Goal

Generate the litter field, record images and measure the camera's coordinate convention. Establish what a timestamp means before using an image to command motion or locate an object.

Predict before running

For a nadir camera over a flat surface, doubling altitude should approximately halve a marker's pixel width. Predict how a marker moves in the image when it moves north or east. Mark which predictions depend on yaw and camera mounting.

Generate and launch

Close the default simulator. In the kit root on Ubuntu, activate its CPU environment. Set `LITTER_PX4` to the actual checkout path; the value below is an example. Derive the kit path from the current directory so spaces remain quoted.

```
export LITTER_KIT="$PWD"
export LITTER_PX4="$HOME/PX4-Autopilot"
python -m integration.prepare_gazebo \
  --px4 "$LITTER_PX4" --out runs/lab07-assets
export LITTER_ASSETS="$LITTER_KIT/runs/lab07-assets"
export LITTER_GZ="$LITTER_PX4/Tools/simulation/gz"
export GZ_SIM_RESOURCE_PATH="$LITTER_ASSETS/models:$LITTER_ASSETS/worlds"
export GZ_SIM_RESOURCE_PATH="$GZ_SIM_RESOURCE_PATH:$LITTER_GZ/models"
export GZ_SIM_RESOURCE_PATH="$GZ_SIM_RESOURCE_PATH:$LITTER_GZ/worlds"
cd "$LITTER_PX4"
PX4_SYS_AUTOSTART=4001 PX4_SIM_MODEL=gz_x500_litter \
  PX4_GZ_WORLD=litter ./build/px4_sitl_default/bin/px4
```

In a second Ubuntu terminal, enter the kit root again. Use the Python environment containing Gazebo Transport 13, Messages 10 and Pillow:

```
gz topic -l
gz topic -i -t /litter_camera/image
python3 -c "from gz.transport13 import Node; from gz.msgs10.image_pb2 import Image"
python3 -m integration.record_camera --out runs/lab07-camera --seconds 10
```

Follow `docs/PX4.md` and the Gazebo Python source link for binding installation. Installing Pillow in one environment does not install Gazebo bindings in it.

Measure four properties

1. Confirm readable RGB 640×640 frames with visible targets. The sensor requests 10 Hz; measure achieved simulation-time intervals and count gaps rather than requiring exactly 100 received frames.
2. Check that `sim_time_s` increases. Keep host arrival time separate from simulation time.
3. At a verified stationary pose, move a labelled marker north then east. Repeat at a changed yaw. Save the images and record the camera-to-body axes.
4. Repeat at two known heights and compare marker widths. Include measurement uncertainty and actual dimensions.

Use the simulator controls or a lab-reviewed hover procedure for stationary measurements. The recorder itself does not hold the vehicle. Its images have no synchronized capture pose yet.

Troubleshooting and completion

No images: inspect topic names, the Sensors plugin and render logs. Wrong axes: inspect the camera link, sensor pose and yaw convention. The toy north-up projection is insufficient when those assumptions change.

Pass with a labelled axis image, a height/scale comparison, timestamp statistics and a written transform convention. Mac-only XML inspection passes preparation only. Keep generated evaluator truth out of the policy loader.

Lab 8 Validate commands and build the observation bridge

Where: isolated Ubuntu SITL host; offline contract design on Mac. **Before starting:** Labs 6–7.

Theory: guide pages 32–33. **Status:** square script supplied; synchronized bridge and independent supervisor remain implementation work.

Goal

First demonstrate position tracking, then define and test the missing connection between camera observations and flight commands. Keep those two completion milestones explicit.

Predict before running

If inference freezes but MAVSDK keeps resending the last setpoint, will PX4 necessarily detect an offboard stream failure? Explain why message streaming and fresh model decisions are different signals.

Milestone A Run the square

Run the default or validated custom simulator on the same Ubuntu host as this script, with no physical aircraft connected. In a separate terminal at the kit root:

```
python3 -m venv .venv-flight
source .venv-flight/bin/activate
python -m pip install mavsdk==2.8.4
python -m integration.px4_square --sitr --out runs/lab08-square
```

The script connects to a local simulator, requests 5 m altitude, visits a 2 m square and lands. Each waypoint requires position error below 0.25 m and speed below 0.2 m/s for one second, with a 30-second timeout. These are exercise values.

Inspect `position.jsonl` and the PX4 flight log. Plot measured north/east against `target_ne`, identify each target change, and measure settling and overshoot. Speed is used by the script's reach check but is not saved in its current JSON log; use the PX4 log when analysing it. Do not treat host receive times as image capture times.

Milestone B Implement the bridge

This is an assignment, not an existing command. Create a timestamped state buffer, observation matcher and separate command supervisor. Begin with recorded streams and a fake clock on your Mac, then connect them in SITL.

Each observation must identify its image, capture time, clock domain, interpolated estimated pose/attitude, calibration and task epoch. Each proposal must identify that observation, its capture anchor, action, computation time and expiry. Define and measure the clock mapping; reject out-of-buffer observations. Add the displacement to the capture anchor once, validate the absolute target, and resend that target while valid.

Agree timing tolerances and a hold response with the lab before executing bridge commands. The guide's 0.5 s validity and 20 Hz supervisor are initial hypotheses to test against measured delay and stopping behaviour.

Fault cases and required evidence

Injected fault	Required observation
Frozen image timestamp or stalled inference	Supervisor detects expiry and uses the reviewed response
NaN or out-of-bounds action	Proposal rejected with reason; no invalid target sent
Duplicate proposal	No repeated accumulation of displacement
Old task result after resumption	Task epoch rejection; new task retains authority
Missing pose near capture time	Observation rejected; no guessed pose association
Communication loss	Actual PX4 response matches reviewed offboard-loss configuration

Completion check

Milestone A passes with the square telemetry explained and landing confirmed. Milestone B passes with clock-alignment measurements, capture-anchor evidence and fault logs that include actual vehicle response. A Python `finally` block does not establish an independent watchdog. Policy-controlled SITL requires both milestones.

Lab 9 Verify the SmolVLA interface

Where: CUDA workstation for runtime; Mac for data preparation. **Before starting:** Labs 3–5. **Theory:** guide pages 49–51. **Status:** movement integration supplied; GPU smoke test pending.

Goal

Make one pretrained movement model consume the documented image, instruction and state, then save and reload a usable drone action interface. This lab does not implement mission-event or semantic heads.

Predict before setup

On your Mac, inspect one training sample with `integration.data.records`. Write its seven state values and two action values with units. Explain how raw RGB differs from the tiny learner's colour features. Predict each tensor's shape and range before checking the table below.

Tensor	Shape before model padding	Range or scaling
RGB	$B \times 3 \times 96 \times 96$	Float pixels divided by 255
State	$B \times 7$	Divide by [1, 1, 5, 1, 1, 1, 1]
Action label	$B \times 1 \times 2$	North/east metres divided by 0.5

The internal 32-dimensional padding belongs to the pretrained implementation. Confirm how the pinned model handles padded action dimensions and loss; record any change before comparing runs.

Set up the CUDA environment

Use a separate Python environment compatible with the pinned LeRobot release. The CPU lab's Python 3.14 verification does not establish compatibility with the GPU stack. Have the lab provide the CUDA-compatible PyTorch build. Then follow `docs/SMOLVLA.md`:

```
python -m pip install \
  'lerobot[smolvla] @ git+https://github.com/huggingface/lerobot.git@v0.4.4'
python -m pip install -r requirements.txt
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
nvidia-smi
python -m integration.data --data runs/lab03-data
```

Copy the complete dataset to the workstation at that path, or use its local equivalent consistently. The first policy load can download pretrained weights. Record the resolved model revision and LeRobot commit, not just their names.

Run the smallest useful smoke test

```
python -m integration.train_smolvla --data runs/lab03-data \
  --out runs/lab09-smoke --steps 1 --batch-size 1
python -m integration.evaluate_smolvla \
  --checkpoint runs/lab09-smoke/best --split val \
  --episodes 1 --out runs/lab09-reload
```

Even a one-step run evaluates the full validation loader. Record the first tensor shapes, finite loss, nonzero trainable gradient norm and names of trainable/reinitialized layers using temporary diagnostic logging. Confirm saved processors exist. Reloading for one rollout must produce two finite action components after postprocessing. Failure to navigate after one update is not an interface failure.

Apple GPU status

The supplied trainer and evaluator explicitly use CUDA. Setting an environment variable will not convert them to MPS. A future Mac port must change device selection, processors, synchronization and validation RNG handling, then pass the same gradient/reload checks. No local MPS training result is claimed here.

Completion check and troubleshooting

Save the environment, tensor/gradient record, one-step history, checkpoint and reload result. Import errors require checking the pinned environment. Shape errors require checking the action time dimension. Memory errors start with batch size one. Do not launch longer training until this milestone passes.

Lab 10 Train and evaluate the movement pilot

Where: CUDA workstation; Mac for report analysis. **Before starting:** Lab 9 runtime pass. **Theory:** guide pages 52–55. **Status:** training/evaluation scripts supplied; results must be measured.

Goal

Establish whether the adapted model learns the movement task, select using development evidence and measure behaviour separately from inference latency.

Predict before running

The toy world pauses during inference. Could a model taking two seconds per decision still succeed in a simulated 2 Hz task? Explain why simulation success does not prove it can meet a 0.5-second wall-clock deadline.

Train in stages

First run a small development job:

```
python -m integration.train_smolvla --data runs/lab03-data \
  --out runs/lab10-short --steps 100 --batch-size 2
python -m integration.evaluate_smolvla \
  --checkpoint runs/lab10-short/best --split val \
  --out runs/lab10-short-val
```

If the interface remains correct and the run fits memory, start a fresh 2,000-step job with `--out runs/lab10-dev`. This is a new run, not a resume of the 100-step optimiser state. Evaluate its best checkpoint on validation scenes with the same settings. Do not invent an expected loss or success threshold from the CPU learner's results.

The supplied trainer selects by validation flow loss. Use closed-loop validation to judge whether that selection is useful. Freeze your selection rule before comparing several checkpoints or configurations. A successful tiny overfit experiment can diagnose wiring; it is not a generalisation result.

Inspect the training bundle

Retain `history.json`, `best/`, processor files and `drone_interface.json`. Also save `python -m pip freeze`, the GPU record, resolved model revision, elapsed time, peak memory, seed and dataset identity. The script does not automatically capture all of these.

Explain which projections were reset, which parameters trained and why the original robot-arm normalisation was replaced. Confirm that a freshly loaded checkpoint produces correctly scaled actions.

Evaluate the frozen model

```
python -m integration.evaluate_smolvla \
  --checkpoint runs/lab10-dev/best --split test \
  --out runs/lab10-test
python -m integration.evaluate_smolvla \
  --checkpoint runs/lab10-dev/best --split test --language swapped \
  --out runs/lab10-swapped
python -m integration.evaluate_smolvla \
  --checkpoint runs/lab10-dev/best --split test --language empty \
  --out runs/lab10-empty
python -m integration.lab_tools compare \
  runs/lab10-test runs/lab10-swapped runs/lab10-empty
```

Compare a classical run on the same split and starts. Record stochastic inference settings; the supplied evaluator does not expose a seed flag, so paired repeated evaluations need an explicitly controlled RNG protocol before research claims about small differences.

Completion check

Save a reloadable checkpoint, matched outcomes, one failure explanation and p50/p95 inference times. The current timing includes cold calls and excludes simulator motion during inference. Report it as an offline timing diagnostic. Device deployment needs a separate real-time experiment. Multiple training seeds are a research extension; report the number actually run.

Troubleshooting

For weak performance, inspect scaling, image content, labels and trainable parameters before increasing model size. If GPU access is unavailable, complete the comparison-table design and mark training pending; CPU teaching weights cannot substitute for SmolVLA evidence.

Lab 11 Build reliable detours and resumption

Where: Mac for milestone A; validated SITL bridge for B. **Before starting:** Lab 5 for A; Lab 8B for B. **Theory:** guide pages 35–42. **Status:** research implementation assignment.

Goal

Build ordinary software that preserves unfinished coverage, validates proposed events and resumes correctly after inspection. Begin with scripted proposals so model errors cannot hide manager errors.

Predict before implementing

A MARK request commits a map entry, then the process crashes before RESUME. What must be stored so restarting and replaying the same request cannot create a second entry or skip survey ground? Why is category alone an insufficient object identity?

Milestone A Implement a replayable manager

Create a new module such as `mission_manager.py`. These are proposed interfaces, not supplied functions:

```
result = manager.accept_or_reject(proposal, observation, now)
# result: accepted event, next state, execution request, reason
snapshot = manager.snapshot()
manager = MissionManager.restore(snapshot)
```

Use a fake clock in tests. Store route version, segment index, progress within the segment, measured coverage, active candidate, map records, context version and task epoch. Include the accepted-event results needed for duplicate handling. Keep actual flight execution outside this module.

Define the transition contract

State	Request	Accepted effect
SURVEY	CONTINUE	Continue the ordinary route
SURVEY	DETOUR	Snapshot progress; select an image-grounded candidate; enter INSPECT
INSPECT	CONTINUE	Validate a bounded inspection movement
INSPECT	ANALYSE	Request a validated hold; enter ANALYSE
ANALYSE	MORE_VIEW	Return to INSPECT with the active candidate
ANALYSE	MARK	Commit evidence and geometry once; remain in ANALYSE
INSPECT or ANALYSE	RESUME	Close inspection; rejoin unfinished coverage; enter SURVEY
Any active state	HOLD	Preserve mission progress and request the reviewed hold

Define recovery out of HOLD explicitly through a validated recovery path. A generic CONTINUE must not accidentally release a fault hold. Define task-epoch increments for mode/task changes and require new proposals to use the current context.

For an already accepted event ID with identical contents, return its stored result without replaying side effects. Reuse of an ID with different contents is an error. For new IDs, validate observation identity, epoch, context, expiry, allowed transition, budget and bounds. Persist the state change, event result and execution intent atomically, then publish a deduplicated execution request.

Test faults before flight integration

- Replay DETOUR and MARK: unchanged snapshot and one map entry.
- Deliver an old movement after RESUME: rejection, no inspection target restored.
- Interrupt after map commit: restart preserves the entry and unfinished route.
- Use two objects of the same category: separate spatial/evidence identities.
- Detour halfway through a segment: coverage, not merely waypoint index, determines resumption.
- Lose the target, camera or valid pose: defined hold/recovery with reasons.
- Exhaust budget or propose an invalid transition: explicit rejection/override record.

Milestone B Close the simulator loop

Connect execution requests to the Lab 8B supervisor. Run a scripted survey, detour, inspection, analysis, MARK and explicit RESUME. Verify the actual path and coverage against the saved snapshot. Include timing, rejected events and physical response in the log.

Completion check

A passes with deterministic replay, restart/fault tests and before/after state records on your Mac. B requires a simulated mission with an evidence-linked map and no skipped unobserved ground. Save a coverage plot. Neither milestone requires a learned event selector yet; report scripted decisions explicitly.

Lab 12 Build and evaluate the unified mission model

Where: Mac for schema/tests; training workstation and SITL for completion. **Before starting:** Labs 10 and 11B. **Theory:** guide pages 46–56 and 66–68. **Status:** research implementation assignment; shared mission heads are not supplied.

Goal

Extend one shared model with mission events, candidate grounding and semantic labels alongside inspection actions. Demonstrate that its accepted decisions improve complete missions while preserving coverage and execution constraints.

Predict before implementing

Can the same visible candidate require different decisions when time is short or it is already mapped? Can an object receive its later close-up label at an earlier ambiguous frame without leaking future evidence? Write paired examples and justify their labels.

Milestone A Build the mission data contract

Create `mission_dataset.py` with an explicit allowlist of deployable inputs and per-head validity masks. Review `templates/single_vla/` as illustrative records; its image path does not resolve and these examples are not training data. Collect separate complete missions with survey, inspection, analysis, resumption, negative candidates and interruptions.

Split whole missions by layout and collection session before sampling frames. Keep evaluator truth and future outcomes separate. Record the observation, task epoch, context version, calibration, proposed event, accepted event and override reason. Include ambiguous labels as unknown where visible evidence is insufficient.

Milestone B Add heads and prove the losses work

Create `unified_policy.py` around one shared backbone. Return event logits, one grounded region, semantic outputs and an inspection action when enabled. Define the units, vocabularies, shapes and validity of every output before training.

Create `train_unified.py` with event classification, region and semantic losses plus the native flow-matching action objective. Normalise each head's loss by its valid labels. A head with no valid examples contributes zero; select valid rows before computing its loss so missing/NaN labels cannot contaminate the batch. Ordinary survey-route commands do not train the inspection-action head.

Test masked labels, all-empty masks, padded dimensions and gradients through shared features. Overfit 10–20 reviewed snippets first, retaining a movement regression check. This milestone diagnoses implementation; it does not measure unseen mission success.

Milestone C Integrate proposals with the manager

Create `evaluate_mission.py` around the Lab 11 manager and Lab 8 bridge. Keep model proposals, accepted events, execution and observed coverage in separate log fields. Allow MARK only with valid evidence and a geometric location from calibration and capture pose. Keep one consistent event per accepted proposal and log every fallback.

Test paired inputs: change the requested category, remaining budget or mapped-object history while holding the image fixed. Use examples whose expert decision should change, and examples whose decision should remain stable. Record both expected and actual outputs.

Milestone D Compare complete systems

Condition	Purpose
B0	Tuned classical detector, fixed decisions and classical movement
B1	Unified learned perception/semantic/action outputs with fixed event rules
U	Complete unified model with learned events
U without context	Measure the contribution of mission history and coverage inputs

Hold routes, layouts, mission clock, action limits, geometry, map matching and budgets fixed. Use matched missions containing absent litter, duplicate sightings, two same-class objects, unsuccessful inspections and partial-segment detours.

Declare map matching tolerance and success criteria before final testing. Report map precision/recall, location error, coverage, detour/hold time, resumption failures, invalid proposals, overrides and camera-to-command age. Count failed and timed-out missions. Select on validation missions; evaluate frozen configurations on grouped test missions. Report actual training seeds and uncertainty across missions.

Completion check and scope

Save the dataset/version contract, shared checkpoint, per-head diagnostics, manager tests, complete mission traces and B0/B1/U comparisons. Passing requires correct simulated execution and resumption on unseen missions. Orin timing and real transfer remain later experiments. If resources stop at an earlier milestone, report that milestone explicitly rather than claiming the unified mission system is complete.

Lab evidence worksheet

Copy this file to `notes/labNN.md`. Replace the prompts with your own observations; leave unperformed checks marked pending.

Identity and status

- Lab number and date:
- Status and environment used:
- Prerequisites passed and their evidence paths:
- Code commit or source snapshot; changed files:
- Python, dependency and platform record:

Prediction

What do you expect before running the experiment, and why? Name the variable you will change and the variables you will hold fixed.

Execution

Record the exact command, configuration, seed, split and output directory. Record failed attempts as well as successful ones. Do not include passwords or access tokens.

Evidence

Link the raw log, images or replay, result summary and relevant tests. Include one concrete value with its unit. For GPU work, retain the resolved model revision, package versions, device, runtime and memory measurement.

Interpretation

Explain one observation using an image, log row or computation. Did it match the prediction? Distinguish the requested action, accepted command and measured response.

Failure and correction

Record one observed or deliberately injected failure, the expected handling and the actual handling. State whether the correction changed the data, software, protocol or environment.

Completion

- Evidence satisfies the lab's pass criteria:
- Remaining limitation or untested assumption:
- What this result supports:
- What still needs a different experiment:
- Next lab or unresolved blocker:

Reference sources

The external pages support platform setup and interfaces. The lab steps and assessment criteria are specific to this teaching package. Check the pinned versions before using examples from newer documentation.

- [PX4 v1.16 Ubuntu setup](#)
- [PX4 v1.16 macOS setup](#)
- [PX4 v1.16 Gazebo](#)
- [Gazebo Transport 13 Python](#)
- [LeRobot v0.4.4 source](#)
- [SmolVLA documentation](#)
- [Apple PyTorch and MPS](#)